



CCIHunter: Enhancing Smart Contract Code-Comment Inconsistencies Detection via Two-Stage Pre-training

ZIWEI LI, Sun Yat-sen University, China

JIAJING WU, Sun Yat-sen University, China

ZHIYING WU*, Sun Yat-sen University, China

DONGCHENG TAN, Sun Yat-sen University, China

WEIPENG ZOU, Sun Yat-sen University, China

ZIGUI JIANG, Sun Yat-sen University, China

YI ZHEN, Sun Yat-sen University, China

ZHENG ZHANG, Nanjing University of Aeronautics and Astronautics, China

ZIBIN ZHENG, Sun Yat-sen University, China

Smart contracts are self-executing computer programs on blockchains. With the development of blockchain technology, the number of smart contracts has grown rapidly, as has the concern for their security. Regrettably, inconsistencies between the logic implemented in the code and the intentions described in the comments, known as Code-Comment Inconsistencies (CCI), are frequently present in some smart contracts. These inconsistencies can mislead readers in understanding the contract code and, in severe cases, may lead to vulnerabilities and economic losses. Existing learning-based methods are not tailored for smart contract languages, overlook the issue of insufficient context information caused by comment references and nested intentions, and rely on large-scale labeled data; whereas rule-based methods struggle to accommodate the flexibility with which developers express intentions, often resulting in false positives. To tackle the challenges posed by insufficient context information and the scarcity of labeled data, we introduce CCIHunter, a tool designed to detect CCIs in smart contracts. CCIHunter addresses the issue of insufficient context information during data modeling and incorporates a two-stage pre-training process that does not depend on labeled data to enhance its detection capabilities. Specifically, CCIHunter enhances comments based on templates and models code as a heterogeneous graph based on function calls. It utilizes CodeBERT and UniMp to generate embeddings for comments and code, respectively, and then calculates the similarity between these two embeddings. Consistency is judged by combining code embeddings, comment embeddings, and similarity scores. Notably, CCIHunter undergoes a two-stage pre-training that includes contrastive learning and mutation analysis, aiming to improve its ability to bridge the gap between code and comments and to focus on code elements at different granularities. Experimental

*Corresponding author

The work described in this paper is supported by the National Natural Science Foundation of China (62372485), and the Natural Science Foundation of Guangdong Province (2023A1515011314), Shanghai Committee of Science and Technology, China (23511101000).

Authors' Contact Information: Ziwei Li, lizw27@mail2.sysu.edu.cn, Sun Yat-sen University, China; Jiajing Wu, wujiajing@mail.sysu.edu.cn, Sun Yat-sen University, China; Zhiying Wu, wuzhy95@mail2.sysu.edu.cn, Sun Yat-sen University, China; Dongcheng Tan, tandch3@mail2.sysu.edu.cn, Sun Yat-sen University, China; Weipeng Zou, zouwp3@mail2.sysu.edu.cn, Sun Yat-sen University, China; Zigui Jiang, jiangzg3@mail.sysu.edu.cn, Sun Yat-sen University, China; Yi Zhen, iamzhenyi@gmail.com, Sun Yat-sen University, China; Zheng Zhang, he1pa404@gmail.com, Nanjing University of Aeronautics and Astronautics, China; Zibin Zheng, zhzhbin@mail.sysu.edu.cn, Sun Yat-sen University, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/8-ART

<https://doi.org/10.1145/3764867>

results demonstrate that CCIHunter achieves a precision of 0.95, a recall of 0.90, and an F1 score of 0.93, outperforming existing tools.

CCS Concepts: • **Software and its engineering** → *Maintaining software*; **Software testing and debugging**.

Additional Key Words and Phrases: Smart Contract, Code-comment Consistency

1 INTRODUCTION

Smart contracts are computer programs that automatically execute on a blockchain. They are extensively utilised in the creation of decentralised applications (DApps), including but not limited to gaming platforms, cryptocurrencies, and decentralised finance. With the development of blockchain and the popularity of DApps, the number of smart contracts has exceeded 27 million [3], leading to a heightened concern for the security of smart contracts on blockchain.

Like programs in other languages such as Python and Java, smart contracts contain code and comments describing the code. Despite developers being aware of smart contracts' immutability once put on the blockchain and taking great care in their implementation, there are nevertheless Code-Comment Inconsistencies (as we call CCIs in this paper) in smart contracts. CCI refers to a misalignment between the logic implied by the contract code and the descriptions provided in the comments. On one hand, CCIs hinder readers from accurately understanding the code, affecting team collaboration [12]. On the other hand, unlike typical syntax errors, CCIs may not result in error messages during execution, making them easy to overlook.

The detection of CCIs in smart contracts requires urgent attention. *First*, smart contracts encompass financial attributes, directly relating to digital assets and transaction execution. The presence of CCIs may indicate that there are defects in the implementation of the code, which could potentially lead to economic losses. For example, in February 2022, Rigo was hacked [1], resulting in the theft of tokens (digital assets on blockchain) valued more than \$432,000. While the comments stated that the "setMulAllowances" method should only be used by the owner, the method lacked the necessary access restriction. Attackers were able to misuse this function to steal tokens by exploiting this inconsistency. *Second*, once smart contracts are deployed on the blockchain, they are difficult to alter. Hence, effective detection of CCIs before deployment is imperative to minimize potential losses. *Third*, code duplication and reuse [10] are prevalent in smart contracts. Developers often omit updating corresponding comments when modifying code to suit their needs [25, 27], easily leading to CCIs in smart contracts.

Current research can be categorized into rule-based approaches [15, 30, 31] and learning-based approaches [21, 22]. **Rule-based approaches** leverage text mining and expert rules to craft specific templates for particular CCIs, extracting information from comments and code to verify their consistency. Although these methods have achieved commendable results in detecting certain CCIs, they necessitate the formulation of targeted rules, which requires specialized expert knowledge. Moreover, the scope of CCIs that these methods can detect is limited, as they cannot identify inconsistencies beyond the established rules. **Learning-based approaches** embed comments and code into vectors for detecting CCIs within the vector space. However, existing learning-based methods are designed for traditional programming languages and cannot be directly applied to smart contract languages. Furthermore, the effectiveness of these methods usually relies on extensive training with labeled data. In the domain of smart contracts, there is a scarcity of research for CCIs, and large-scale labeling is virtually non-existent. The lack of labeled data prevents existing learning-based methods from being adequately trained, resulting in suboptimal performance in detecting CCIs in smart contracts.

Detecting CCIs in smart contracts faces two primary challenges: *Firstly*, the quality of comments within smart contracts is highly variable, introducing the problem of insufficient contextual information. Developers may not directly express or implement their intentions, often employing methods such as *comment referencing* and *intent nesting* to articulate their objectives. Such practices can lead to the problem of insufficient contextual information, potentially causing false positives or negatives in the detection of CCIs (which will be detailed in the motivating

examples). *Secondly*, there is a paucity of large-scale labeled data for CCIs within the smart contract domain. Although the use of Solidity for writing smart contracts is on the rise, the ecosystem is not as mature as that of traditional programming languages. Moreover, research on CCIs specific to smart contract is scarce. Currently, only SmartCoCo [15] is dedicated to detecting CCIs in the Solidity ecosystem—resulting in a lack of extensive labeled data for use in such studies.

Specifically, comment referencing occurs when developers cite function comments from other standard contracts as the comments for their own functions, often using phrases such as “See ***”; intent nesting refers to the practice where one function calls another, and the caller’s intent mentioned in the comments is implemented in the called function. Due to the prevalent code reuse in smart contracts, both comment referencing and intent nesting are common practices. Developers routinely rely on the comments of reused functions and prefer to achieve their objectives by invoking commonly used business functions. In particular, we collect 16 open-source projects from the top 50 Ethereum DApps on the DAppRadar [2] leaderboard and find that among the 16 projects, there are 1,221 commented functions, with 329 (26.95%) functions utilizing comment referencing. Additionally, Hao et al. [15] has shown that in a collection of 39,372 smart contracts, over 60% of function constraints mentioned in comments are implemented within another function, a phenomenon we define as intent nesting.

To address the aforementioned challenges, we propose CCIHunter, a code-comment inconsistency detection model based on a two-stage pre-training approach. On the one hand, CCIHunter takes into account the issue of insufficient contextual information brought by comment referencing and intent nesting. It implements template-based comment enhancement and function call-based AST expansion during data modeling, injecting richer contextual information into code and comment data. On the other hand, considering the lack of large-scale labeled data in the Solidity language, CCIHunter undergoes a two-stage pre-training process that includes contrastive learning and mutation analysis. This pre-training process is unsupervised, relying not on labeled data, and enhances the model’s ability to detect CCIs. Note that our proposed method is primarily aimed at smart contracts written in the Solidity, but our approach can be extended to other languages.

CCIHunter is designed for smart contract developers with the aim of assisting them in uncovering potential CCIs. It could be used to detect CCIs before developers deploy smart contracts to the blockchain, providing alerts to avoid potential losses. To facilitate further research, our code is freely available online¹. Overall, our major contributions are as follows:

- To the best of our knowledge, we are the first to propose a learning-based CCI detection method for smart contracts written in Solidity, offering new insights into the field.
- We introduce CCIHunter, a code-comment inconsistency detection model that leverages data augmentation and a two-stage pre-training process. CCIHunter does not rely on a large amount of labeled data and is adept at handling the issue of insufficient contextual information.
- Experimental results indicate that CCIHunter has achieved an F1 score of 0.93 in the task of detecting inconsistencies between smart contract code and comments, representing an relative improvement of 86% over the current state-of-the-art techniques.

The rest of this paper is organized as follows. In the second section, we discuss related work. The third section presents motivation examples to facilitate a better understanding of our work. The fourth section details CCIHunter, including its design and training procedures. In the fifth section, we report our experiments and discuss the threats to validity. In the sixth section, we engage in some discussions and hope to share our insights with researchers and smart contract developers. Finally, the seventh section provides a summary of our research.

¹<https://zenodo.org/communities/ccihunter>

2 RELATED WORK

In this section, we discuss related work on code-comment co-evolution, detecting comment-code inconsistencies, and descriptions of code intentions.

2.1 Code-comment co-evolution

Co-evolution of comments and code refers to the synchronized evolution of code and corresponding comments, highlighting their interdependence in the software development process. Previous work has explored the issue of code-comment co-evolution from various perspectives. Wen et al. [32] conducted an empirical study to investigate the possibility of various kinds of code modifications causing comment updates and defined a taxonomy of the code-comment inconsistencies fixed by developers. Fluri et al. [14] investigated whether developers comment on their code and to what extent they add comments or adapt them when they evolve the code. Ibrahim et al. [16] studied comment update practices in three large open-source systems written in C and Java. Their findings suggest that a change in which a function and its comment are suddenly updated inconsistently is risky and should be reviewed carefully. Corazza et al. [11] created a publicly available dataset by manually evaluating the consistency between the comments and implementations of 3,636 Java methods. They revealed that when comments and implementations are consistent, there is a higher lexical similarity between the comments and the code. Arafat et al. [8] took 5,229 open-source projects written in different programming languages as research samples and focused on analyzing the comment density, which is a key indicator defined as the ratio of the number of comment lines to the number of code lines. They found that comment density is closely related to programming languages. Specifically, Java systems exhibit a remarkable comment density, reaching up to 25% at most.

The aforementioned studies have conducted investigations into the co-evolution between code and comments, which can offer significant direction for the creation of instruments intended to identify and address CCIs.

2.2 Detecting comment-code inconsistencies

Numerous methods have been developed to detect different types of CCIs across programming languages. Many relied on rule-based approaches, utilizing predefined rules, natural language processing (NLP), and static program analysis. For instance, iComment [28] automatically analyzes natural language comments to extract implicit program rules for CCI detection. @tcomment [31] inferred attributes from Javadoc comments and method code to detect CCIs related to null values. SmartCoCo [15] identified comment constraints and performed lightweight program analysis to detect three types of CCIs in smart contracts. Additionally, Fraco [24] and Hotcomments [29] focused on detecting CCIs related to identifiers and interrupts, respectively. However, rule-based methods often focus on comments related to specific code attributes or types, thus these methods are usually inapplicable for detecting other types of CCIs. Besides, the design of rules is a time-consuming task which demands extensive analytical efforts.

Another kind of relevant work involves learning-based methods. For example, Rabbi et al. [21] introduced a topic modeling-based approach. It utilized topic modeling to extract features from both code and comments, merged consistency scores from different random forests, and thus judged their consistency. Rabbi et al. [22] also detected CCIs using a siamese recurrent network which utilizes word tokens in codes and comments as well as their sequences in corresponding codes or comments. However, the lack of large-scale labeled data in smart contracts may hinder the effectiveness of some learning-based methods.

Note that among the existing work, only SmartCoCo [15] is designed for smart contracts. As mentioned in the Introduction, current research on CCI detection for other programming languages can be categorized into rule-based and learning-based approaches. Among them, rule-based methods are not applicable to smart contracts because their rules are related to the syntax of the target language. As for learning-based methods, they do

not take into account the special challenges in the context of smart contracts. Directly applying them to smart contract detection may not yield good results.

2.3 Detecting outdated comments

Another type of work related to the detection of code-comment inconsistencies is the detection of outdated comments, also known as just-in-time code-comment inconsistency detection. It refers to the detection of situations where comments and code become inconsistent due to code modifications. For example, Xu et al. [34] introduced MCCL. They employed an attention mechanism to connect code and comments, identifying how code changes impact comments. They also designed a confidence learning denoising component to remove feature noise and label errors, improving dataset quality. Sheena et al. [20] proposed a deep-learning approach which can learn to associate comments with code changes to detect whether comments are inconsistent with the code after code changes. Liu et al. [18] presented a machine-learning method. By analyzing code change differences, comment content, and their relationships, using 64 features, it can detect outdated comments caused by code updates.

These studies focus on code-comment inconsistencies caused by code changes, and thus require an analysis of code changes during the process. In contrast, we solely concentrate on the code itself, and the input of our method does not involve code change records.

2.4 Descriptions of code intentions

Besides comments, researchers have looked into other intent descriptions for intent consistency checks, like front-end views and API documentation. Zhou et al. [35] proposed an approach to identify inconsistent behaviors of DApps on EVM-compatible blockchains by contrasting the behaviors of DApps that derived from the front-end, blockchain wallet, and smart contracts, respectively. Zhu et al. [36] proposed a technique, DocCon, to detect inconsistencies between documentations and the corresponding code for Solidity.

However, in real-world scenarios, code and its corresponding API documentation and views are often not produced synchronously. In some cases, certain code may not even have corresponding API documentation or views. Previous studies [19, 28, 35] evaluated the quality of different intent descriptions. They suggested that among various descriptions of code intentions, comments show an intuitive relationship to the code and can effectively describe the business intent of the codes. Therefore, in this work, we choose to use comments as the developer intent description for CCI detection.

3 MOTIVATING EXAMPLES

To more effectively demonstrate the motivation of our work, highlight the effects of comment referencing and intent nesting, and show the flexibility of developers' expression of intentions, we present several examples in this section.

3.1 The Impact of Comment Referencing and Intent Nesting on CCI Detection

Example 1 (Fig.1) shows an instance of intent nesting. In Example 1, the function "transferFrom" calls the function "safeSub", and the comment mentions that "The from account's balance is less than the transfer quantity." being one of the exceptional conditions, which implies that the balance should be greater than or equal to the transfer quantity. This intent is implemented in the called function by checking the relationship between the input "allowance[from][msg.sender]" and "quantity". If one does not carefully consider the code of the called function, it might be assumed that the "transferFrom" function does not implement the check of "balance". In fact, this is a case in SmartCoCo [15], where, despite implementing some fact propagation, it fails to accurately extract

```

1 /**
2  * @notice ERC20 transferFrom function; transfers `quantity` tokens from
3  *       `from` to `to` if the sender is approved.
4  * @param from The spender; balance is deducted from this account.
5  * @param to The recipient.
6  * @param quantity The quantity to transfer, in wei.
7  * @dev Exceptional conditions:
8  *       * The contract is paused if it is not self-destructing.
9  *       * The `from` account has approved the sender to spend less than the transfer
10         quantity.
11         * The `from` account's balance is less than the transfer quantity.
12         * The `to` parameter is 0x0.
13 * @return Always returns true if no exception was thrown.
14 */
15 function transferFrom(address from, address to, uint quantity) public
16     pausableIfNotSelfDestructing returns (bool){
17     // safeSub handles insufficient allowance.
18     allowance[from][msg.sender] = safeSub(allowance[from][msg.sender], quantity);
19     return _transfer(from, to, quantity);
20 }
21
22 /**
23  * @dev Returns the difference of the given arguments. Will throw an exception iff `x < y`.
24  * @return `y` subtracted from `x`.
25 */
26 function safeSub(uint x, uint y) pure internal returns (uint){
27     require(y <= x, "Safe sub failed.");
28     return x - y;
29 }

```

Fig. 1. Example1: 0x4014aabaed9923818899af6ae5546aeb254d9fe6

the right code constraints and code fact that the called function “safeSub” checks the amount of balance, thus misjudging the function as exhibiting a CCI.

In Example 2 (Fig.2), the “transferFrom” function within the GravisCollectible contract contains a comment reference to the homonymous function of IERC721. In such cases, it is necessary to find the corresponding function intent within the comments of the referenced function (lines 2-5). Additionally, similar to Example 1, this function also achieves its primary intent through the called function “_transfer”. Despite this, the check mentioned in the comment regarding whether the “from” address is the zero address is not implemented in either the function or the called function. Ignoring similar comment references may lead to overlooking potential code constraints, resulting in false negatives during CCI detection.

3.2 The Flexibility of Developers in Expressing Intentions

The way developers describe and implement their intentions is flexible. For example, developers may implement function call permissions not only through modifiers but also via require statements or if statements. Similarly, for parameter constraints, developers might use if statements in addition to require statements. In terms of comments, for example in Example 3 (Fig. 3), developers do not directly state in the comments what event needs to be emitted,

```

1 interface IERC721 is IERC165 {
2     /**
3      * @dev Transfers `_tokenId` token from `_from` to `_to`
4      * - `_to` cannot be the zero address...
5      */
6     function transferFrom(address from, address to, uint256 tokenId) external;
7 }
8
9 contract GravisCollectible is GravisCollectionMintable, Whitelist, ERC165, IERC721... {
10 ...
11     // @dev See {IERC721-transferFrom}.
12     function transferFrom(address _from, address _to, uint256 _tokenId) public virtual
13         override {
14         require(_isApprovedOrOwner(_msgSender(), _tokenId), "ERC721: transfer caller is not
15             owner nor approved");
16         _transfer(_from, _to, _tokenId);
17     }
18
19     /**
20     * @dev Transfers `_tokenId` from `_from` to `_to`.
21     * As opposed to {transferFrom}, this imposes no restrictions on msg.sender.
22     * Emits a {Transfer} event.
23     */
24     function _transfer(address _from, address _to, uint256 _tokenId) internal virtual {
25         require(isOwner(_from, _tokenId), "ERC721: transfer of token that is not own");
26         require(_to != address(0), "ERC721: transfer to the zero address");
27         _approve(address(0), _tokenId);
28         (uint256 index, uint256 mask) = _position(_tokenId);
29         tokens[_from].data[index] &= ~mask;
30         tokens[_to].data[index] |= mask;
31         owners.add(_to);
32         owners.remove(_from);
33         emit Transfer(_from, _to, _tokenId);
34     }
35 }

```

Fig. 2. Example2:0x3fe15e14187074f21dc5d354cb70695de014a562

but rather say that a “corresponding event” needs to be emitted. In Example 1, developers do not directly describe the constraints of the function, but list the exceptional conditions of the function. The flexibility of developers in expressing and implementing their intentions is difficult to address with relatively rigid rules. Existing work on detecting CCIs in smart contracts is primarily carried out by SmartCoCo [15], which extracts comments and code facts based on pre-set templates and then compares these extracted facts to identify inconsistencies. However, due to the flexibility with which developers express and implement their intentions, and the relative rigidity of the rules, rule-based methods can lead to a certain number of false positives. For example, in Example 3 (0x3fe15), SmartCoCo generated a false positive because the extracted code constraint was “emit a corresponding event,” while the code fact was “emit ContractAddressUpdated event.” Moreover, for function call permissions,

```

1  /// Updates the contracts address and emits a corresponding event
2  function setContract(string calldata contractName, address addr, bool managedContract)
    external override onlyAdminOrMigrationManager {
3      require(!managedContract || addr != address(0), "managed contract may not have address(0)");
4      removeManagedContract(contracts[contractName]);
5      contracts[contractName] = addr;
6      if (managedContract) {
7          addManagedContract(addr);
8      }
9      emit ContractAddressUpdated(contractName, addr, managedContract);
10     notifyOnContractsChange();
11 }

```

Fig. 3. Example3:0x3c68dfc45dc92c9c605d92b49858073e10b857a6

SmartCoCo focuses solely on modifiers; for parameter constraints, SmartCoCo's templates cannot extract code facts from if statements. We have analyzed all false positive examples of SmartCoCo and found that 19 examples (20.88%) were due to the inflexibility of the rules, which failed to correctly extract the code facts. Therefore, in our work, we choose a learning-based approach rather than a rule-based approach for CCI detection.

3.3 The Characteristics of CCIs

Upon examining the CCIs within smart contracts, we have distilled two characteristics associated with CCIs. First, there is a language gap between smart contract languages and natural languages. This is attributed to the inclusion of unique domain knowledge of decentralized applications within smart contracts. An example is the association between the decrease in balance and the term “burn” in the smart contract domain. In traditional financial industries, a reduction in balance might correspond to other terms such as “withdrawal.” In other contexts, the decrease in balance and the concept of “burn” are rarely directly linked. Lack of understanding of DApp domain knowledge by the model may lead to misunderstandings, thereby influencing judgment outcomes. Second, the elements that cause CCIs can be quite minute, ranging from statements to variables or even operators. For example, changing the $\leq$ to $\geq$ in the require statement of Example 1 can completely alter the logic of the code, leading to CCI. Additionally, if the comment emphasizes the emission of a specific event, but the corresponding emit statement is missing in the code, it would also lead to CCI, which is a common cause of CCI. The model needs to be sensitive to a variety of fine-grained code elements in order to effectively detect CCIs.

In summary, we have observed that it is necessary to account for the issue introduced by comment references and nested intentions when detecting CCIs. Moreover, existing rule-based methods struggle to cope with the flexible expression and implementation of developers' intentions. Consequently, we introduce CCIHunter, a pre-training-based method for detecting CCI in smart contracts. Compared to previous approaches, our method: (1) Considers comment references and nested intentions during the data modeling process, enriching the context by injecting additional information through data augmentation. (2) Does not rely on rule templates; instead, it trains a machine-learning model for detecting CCIs. (3) Does not depend on large-scale labeled data for model training.

4 PROPOSED METHOD

Motivated by the insights gained from our observations, we propose a pre-trained CCI detection model named CCIHunter. Fig. 4 illustrates the overall framework of CCIHunter. CCIHunter consists of four parts: data modeling, data embedding, model pre-training, and CCI detection. (1) Data modeling: CCIHunter begins with the extraction

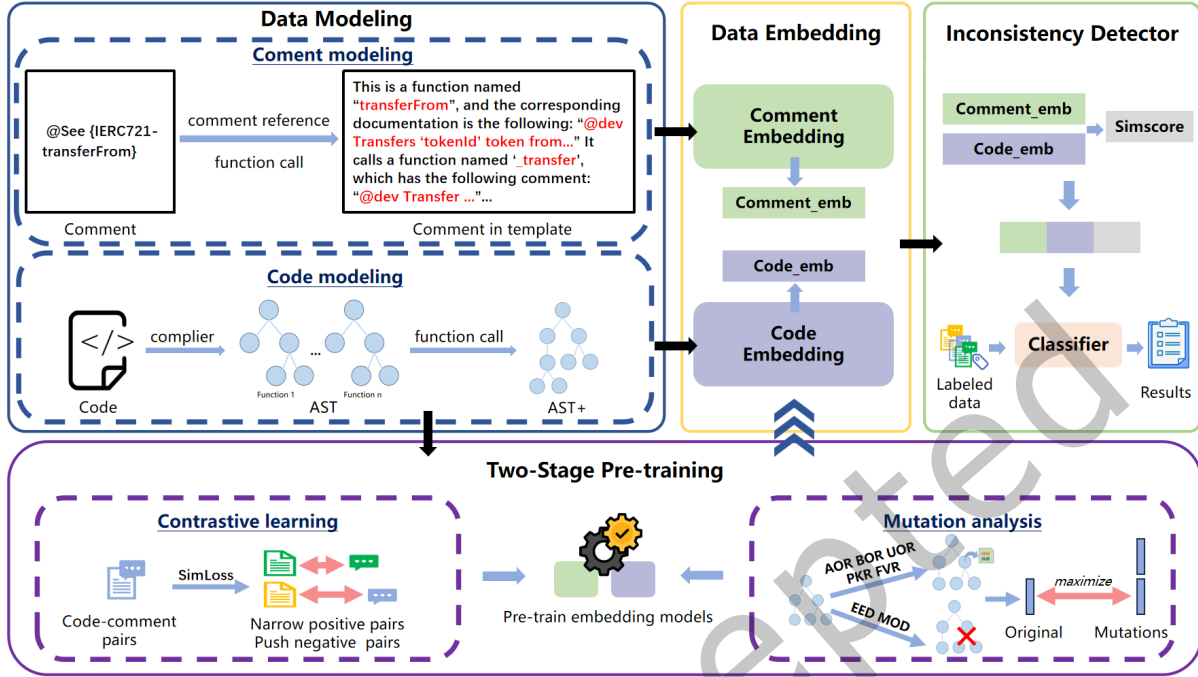


Fig. 4. Overview of CCIHunter

and modeling of comments and code. Comments are encapsulated within a specific template, while the code is transformed into a heterogeneous graph based on the Abstract Syntax Tree (AST). Throughout this process, we handle instances of comment reference and function invocations, enhancing the contextual richness of both the code and the accompanying comments. (2) Data embedding: The modeled data from comments and code are then directed into their respective embedding modules. These modules produce embeddings that capture the intent of the comments and code. (3) Model pre-training: To generate higher-quality embeddings, we design a targeted two-stage pre-training for CCIHunter, which includes a contrastive learning phase and a mutation analysis phase. The contrastive learning phase is designed to enable CCIHunter to learn domain knowledge and bridge the gap between code and comments. The mutation analysis phase is designed to enable CCIHunter to capture changes in code elements at different granularities, thereby allowing it to focus on these code elements. During the pre-training process, the parameters of the two embedding models are adjusted. It is noteworthy that the pre-training process does not require a large-scale labeled dataset. (4) CCI detection: Next, we calculate the similarity score between the two embeddings, merge the code embedding, comment embedding, and similarity score into a single feature vector, and then input this combined vector to a classifier to determine the presence of CCIs.

4.1 Data modeling

CCIHunter takes contracts that include code and comments as input, and outputs the result of CCI detection for functions within the contracts. The first step that needs to be taken is to model the code and comments.

For code, data modeling mainly involves four key steps: (1) Compile the code using the solc compiler. (2) Associate the comments with the AST obtained from compilation. (3) Expand the AST based on function calls to obtain AST+. (4) Extract features from AST+ as the input for the graph neural network model.

(1) Compiling. With the help of the solc compiler [5], we can extract comments (excluding those within the function body) from the contracts and generate the AST of the contracts. An Abstract Syntax Tree is a tree-like structure that represents the syntactic structure and hierarchical relationships of the code. The generated AST contains various types of nodes, such as Block, Assignment, Conditional, FunctionCall, FunctionDefinition, EmitStatement, etc., with different types of nodes corresponding to different granularity and types of code elements in the source code.

(2) Associating code with comments. Although many useful compilation products are provided, we still need to address two issues. The first is the connection between the AST and comments. Comments are at the function level, so to link functions with comments, we need to extract function-level ASTs. We find that there are some nodes in the AST that are of the FunctionDefinition type. The subtree formed by these nodes and their children nodes corresponds to the AST of functions, and the node information includes the function name. Therefore, we can connect the AST of a function with its comments through the FunctionDefinition node and its subtree, utilizing the function name.

(3) Expanding the ASTs. The second issue is the invocation between functions. Although we can extract the AST of each function, the call relationships between functions are not fully considered. As we mentioned earlier, smart contracts have the issue of intention nesting, and the calls between code imply the logic of the code. Therefore, we propose to extend the function AST to generate AST+, thereby more specifically representing function calls on the abstract syntax tree. Specifically, we identify function call nodes in the AST, which are nodes of the FunctionCall type. Function call nodes contain the ID information of the called function, and based on this, we find the AST of the called function and replace the function call node with the root node of the called function's AST, thus achieving AST expansion based on function calls. The called function may also call other functions, and if the expansion is carried out without limits, it may lead to an overly large scale of the expanded AST. To explore the appropriate limit on the number of AST expansions, we conduct research on the 16 open-source DApp projects mentioned earlier and find that among the 14,039 functions surveyed, 34.87% of the functions have function call situations, with 84.50% of the functions having a maximum call depth of 1 level, 11.97% having a maximum call depth of 2 levels, 2.68% having a maximum call depth of 3 levels, and 0.86% having a maximum call depth of 4 levels or more. Therefore, we limit the number of AST expansions to 2 times, which is sufficient to include most of the key function calls.

(4) Feature extraction. The obtained AST+ encompasses nodes of various types, each corresponding to different granularities of code elements, and the types of edges connecting these nodes also vary. Since graph neural networks perform vector calculations, for the convenience of vector computation, we consider extracting and constructing feature vectors based on AST+ to enable the graph neural network to learn the information contained in AST+. During the feature extraction process, we attempt to extract the attributes of the AST as node features or edge features. Attributes that can be numericalized are numericalized, while those that cannot be numericalized are enumerated as one-hot encodings. For each node, we extract the position of the code it corresponds to within the contract as a feature of the node, reflecting the connection between the AST and the source code. Additionally, for nodes related to operators in AST+ (i.e., nodes of types BinaryOperation, UnaryOperation, and Assignment), we construct a one-hot vector as a feature, where each dimension of the vector represents a different operator. Furthermore, for FunctionDefinition nodes, we also extract the function's stateMutability and visibility information, which specify whether the function modifies the contract state, its access level, and whether it can be called externally from the contract. These two features of the function are also represented using one-hot encoding. For edges, we extract the node numbers at both ends of the edge, as well as the sequential number of

Template: This is a function named "**function name**", and the corresponding documentation is the following: **{comment}**; During its execution, it calls a function named "**called function 1 name**", which has the following comment: **{called function 1 comment}**"; it calls a function named "**called function 2 name**", which has the following comment: **{called function 2 comment}**"...

Before enhancement	After enhancement
See {IERC721-transferFrom}	This is a function named ' transferFrom ', and the corresponding documentation is the following: " @dev Transfers 'tokenId' token from 'from' to 'to'... " During its execution, it calls a function named ' _transfer ', which has the following comment: " @dev Transfers ... Emits a {Transfer} event "; it calls a function named ' isApprovedOrOwner ', which has the following comment: " @dev Returns whether _spender is allowed to manage 'tokenId' "; it calls a function named ' _msgSender '

Fig. 5. Comment modeling example

this edge among the edges originating from the parent node as features. These features reflect the relationships between different types of nodes and the sequential characteristics of code elements.

For comments, we design a template to encapsulate the comment information. Templates have been proven by Radford et al. [23] to effectively enhance the performance of models in natural language processing tasks. They provide clear guidance to the model, enabling it to better understand the connection between functions and comments. Determining an appropriate template requires considering two issues: First is comment referencing. In smart contracts, developers often make reference to comments when implementing functions that are similar to or based on standard contracts (such as IERC20, IERC721, etc.). For these comment references, we perform comment recovery, finding the corresponding function comments in the standard contracts and updating the function's comments. Note that the comments which refer non-standard contracts are not currently considered, however, the step of updating the referenced comments is extensible. If there are other commonly referenced contract function comments, they can be easily added to the contract comment library to enable the updating of these comments. Besides, similar to modeling code data, we strive to reflect the function's call relationships in the comment data as well, supplementing contextual information. Based on the function call situations mined from the AST, we also include the information of the called functions and their comments (if available) as part of the comment data. Therefore, we have determined the template as shown in Fig. 5, and provide the comment modeling result of line 11 in Fig. 2 as an example.

4.2 Data embedding

Through data modeling, we encapsulate comments using templates and represent the code as AST+. Next, we input the modeled data into the data embedding module. The code embedding model and the comment embedding model within the embedding module respectively embed the code and comments, thereby obtaining the code embedding vectors and the comment embedding vectors.

For comments, we choose CodeBERT [13], a code-oriented pre-training model developed by Microsoft Research Asia (MSRA), to generate embeddings. CodeBERT demonstrates significant advantages in embedding natural language, such as comments. On one hand, it leverages the Transformer architecture, utilizing self-attention mechanisms to capture nuanced contextual information within comments. On the other hand, CodeBERT is trained on extensive and diverse datasets, attaining robust semantic representation capabilities, and enabling it to recognize and encode complex semantics within comments. Inputting comments into the CodeBERT model yields high-dimensional comment embeddings with rich semantic information.

Regarding code, the AST+ obtained is a heterogeneous graph, preserving categories and properties of different program elements. Consequently, we employ an graph neural network model, i.e., UniMP [26], to generate embeddings for code. UniMP learns the features of nodes and edges in the graph by propagating and aggregating features in the graph, which can generate embeddings for the earlier obtained AST+. Here we choose UniMP due to its capability to simultaneously handle node and edge features. It can effectively extract and learn information within the graph.

4.3 Model pre-training

To enable the two embedding models mentioned earlier to better learn the semantics in the data and generate higher-quality data embeddings, further training is required. We combine the characteristics of CCI in smart contracts and set up a targeted two-stage pre-training, including contrastive learning and mutation analysis.

Compared to consistent code-comment pairs, inconsistent pairs exhibit a larger disparity between comment and code embeddings, manifested as a substantial distance in the embeddings. Therefore, we compute a similarity score (i.e., SimScore) between comment embeddings and code embeddings using the formula in Equ. 1:

$$\text{SimScore} = \frac{v_{code} \cdot v_{com}}{\|v_{code}\| \cdot \|v_{com}\|} = V_{code} \cdot V_{com} \quad (1)$$

where v_{code} and v_{com} are embeddings of code and comments, V_{code} and V_{com} are their respective normalizations. Normalizing the embeddings involves dividing each embedding vector by its own norm, which scales the norm of the embedding vector to 1. This makes different embedding vectors comparable when calculating similarities.

SimScore is the cosine similarity between the two embeddings. It ranges from -1 to 1, with a result closer to 1 indicating that the code and comments are more directionally similar in the feature space. When the SimScore is low, it implies a greater difference between the code and comments in the feature space. Using the similarity score as one of the features helps the classifier determine whether there is an inconsistency. We design the loss function in the pre-training process based on this idea.

4.3.1 Contrastive learning. Contrastive learning, widely employed in the field of deep learning, refers to the representation learning achieved through comparing relative relationships between samples. During the contrastive learning phase, we provide n code-comment pairs in a batch. In the experiments of this paper, we set n to 64, resulting in a total of 15,625 batches. For i -th pair, we formulated it as $P_i(\text{Com}_i, \text{Code}_i)$, where Com_i denotes the comment in the pair, and Code_i means the code in the pair. For each Com_i , its matching positive sample is Code_i , and the remaining code Code_j ($i \neq j$) are considered negative samples. Our objective is to ensure that, for positive samples, the generated embeddings exhibit high SimScore, while for negative samples, SimScore are lower. To achieve this objective, we introduce a loss function called L_1 (Equ. 2) for training.

$$F(a, b) = \exp(V_{com_a} \cdot V_{code_b})$$

$$L_1 = -\frac{1}{2} \sum_{i=1}^n \left[\log \left(\frac{F(i, i)}{\sum_{j=1}^n F(j, i)} \right) + \log \left(\frac{F(i, i)}{\sum_{j=1}^n F(i, j)} \right) \right] \quad (2)$$

where a, b, i and j are indices of code or comment among code-comment pairs, V_{com_i} and V_{code_i} represent the normalized embedding of comment and code for the i -th pair, and n is the total number of pairs.

Through the optimization of the loss function, we adjust the parameters of the code embedding model and the comment embedding model. We guide the two embedding models to bring positive code-comment pairs as close as possible in the embedding space, while simultaneously pushing away negative code-comment pairs. This training process facilitates the model in generating more meaningful embeddings for code and comments, providing a robust pre-training foundation for subsequent work.

4.3.2 Mutation analysis. The concept of mutation analysis originates from widely adopted mutation testing in software testing. Mutation testing is a technique that evaluates the quality of a test suite by making small changes (mutations) to a software program. Considering that code elements at different granularities can lead to CCIs, CCIHunter should have the capability to focus on these various code elements. Therefore, we have established a mutation analysis task aimed at capturing code changes, where the code embedding model is required to increase the distance between the feature space representations of code data before and after the changes. Drawing on related work in Solidity mutation analysis [9], we have designed seven mutation operators, including *Assignment Operator Replacement (AOR)*, *Binary Operator Replacement (BOR)*, *Unary Operator Replacement (UOR)*, *Event Emission Deletion (EED)*, *Modifier Deletion (MOD)*, *Payable Keyword Replacement (PKR)*, and *Function Visibility Replacement (FVR)*, corresponding to different code elements. We select these seven mutation operators because they are related to the nature of functions or the intent of the code. For instance, the deletion of an emit statement may directly lead to the occurrence of CCI, as many developers specify in comments the need to emit certain events for recording, notifying, and monitoring contract behavior; furthermore, mutations such as AOR, BOR, and UOR directly affect the computational results of the code. Operators are a crucial element in implementing the business logic of smart contracts. For example, lines 26-28 in Example 2 contain assignment and unary operators used to implement the business logic of a transfer.

We provide the code embedding model with both the original code data ($Code_0$) and the mutations ($Code_i, i \neq 0$) for learning. The code embedding model generates embeddings for the data before and after mutation. We denote the normalized embedding of the original data as V_0 and the normalized embedding of mutated data as V_i (for $i \neq 0$).

$$p_i = \frac{\exp(V_0 \cdot V_i^T)}{\sum_{j=0}^N \exp(V_0 \cdot V_j^T)}$$

$$L_2 = -\frac{1}{N+1} \left(\log(p_0) + \sum_{i=1}^N \log(1 - p_i) \right) \quad (3)$$

Our optimization objective is to widen the gap between the original code and the mutated code. Therefore, we utilize the commonly employed cross-entropy loss function, typical in classification tasks, as our loss function. The calculation method is defined by Equ. 3, where N represents the number of mutants, and p_i represents the probability calculated by applying the softmax function to the SimScore of between the i -th mutant and the original code. p_i ($i \neq 0$) signifies the likelihood of consistency between the i -th mutated code data and the original code data. With a higher expected value for p_0 and lower expected values for p_i ($i \neq 0$), L_2 computes the sum of the negative log-likelihood of p_0 and the negative log-likelihoods of $1 - p_i$ ($i \neq 0$). By minimizing L_2 , we adjust the parameters of the code embedding model. The code embedding model is guided to create a separation in the embedding space between $Code_0$ and the i -th mutated code data $Code_i$, directing the code embedding model's attention to different code elements.

4.4 CCI detection

Employing the embedding models that have been pre-trained, we can acquire the embeddings for comments and code. Both of the two embedding models generate one-dimensional vectors using the Multi-Layer Perceptron (MLP) layer. Therefore we can subsequently calculate the similarity score between two embeddings using Equ. 1. We concatenate these three features—comment embeddings, code embeddings, and the similarity score—into a single feature, which is then fed into a classifier to determine the presence of CCIs. In our research, we have opted for a random forest classifier. This is because in subsequent experiments, the results show that the random forest classifier is a better choice for this task. Random forest classifier is an ensemble learning method that

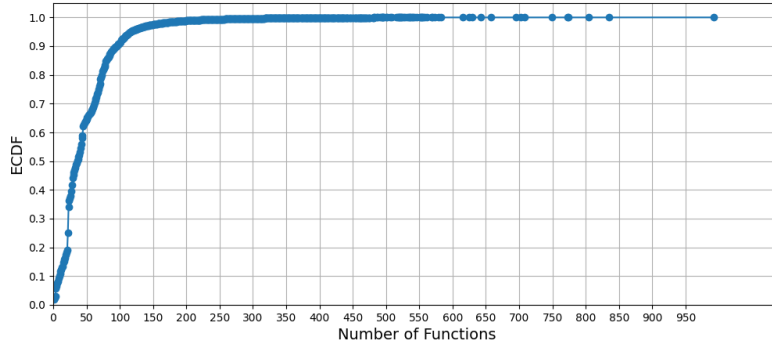


Fig. 6. The empirical cumulative distribution function of number of functions per contract

improves the accuracy and stability of predictions by constructing multiple decision trees and integrating their results. The random forest treats each element as an independent feature and splits these elements during the tree construction. The classifier requires some labeled data for fitting. Therefore we use 80% of the existing labeled dataset [15] as the training set for the classifier fitting.

5 EXPERIMENTS

In this section, we present our evaluation experiments on CCIHunter and analyze the results. In this paper, we present CCIHunter, a detection tool for CCIs based on a pre-training method. **Firstly**, our method includes a classifier. The choice of the classifier and the setting of its parameters will affect the detection results. Therefore, we will explore the better choice of the classifier. **Secondly**, the performance of CCIHunter in detecting real-world smart contracts is an important evaluation dimension. Moreover, considering that some existing tools have also made relevant efforts, we will compare CCIHunter with some baselines to evaluate its practical value. **Thirdly**, as mentioned earlier, our method based on pre-training can effectively reduce the method's dependence on labeled data. Therefore, we will evaluate the degree of CCIHunter's detection performance dependence on labeled data by providing different amounts of labeled data for the classifier to fit. **Fourthly**, the design of CCIHunter has made efforts in data modeling, pre-training, and feature extraction. The effectiveness of each component is worthy of evaluation and discussion. Based on the above motivations, we will explore and answer the following four Research Questions (RQs) in the subsequent experiments.

- **RQ1:** What is the better choice of the CCIHunter classifier and its parameters?
- **RQ2:** How does CCIHunter perform compared with existing technologies?
- **RQ3:** How does CCIHunter depend on labeled data?
- **RQ4:** Is the method design of CCIHunter effective?

Dataset. For pre-training, we use the blockchain data crawler tool BlockchainSpider proposed in our previous work [33] to scrape 3.57 million smart contracts from Etherscan [4] before April 2023. Among them, 682,751 contracts are open-source, and we compile and model them. After removing contracts that fail to compile (due to low-version compilers, compilation timeouts, etc.) and contracts where all functions have no comments, there are 268,586 remaining contracts (approximately 40%), containing 12,992,641 function instances. The mapping of these function instances on the contracts is shown by the empirical cumulative distribution function in Fig. 6. It can be observed that approximately 60% of the contracts contain fewer than 50 functions, and 90% of the contracts have no more than 100 functions. There are also a small number of contracts that contain several hundred commented

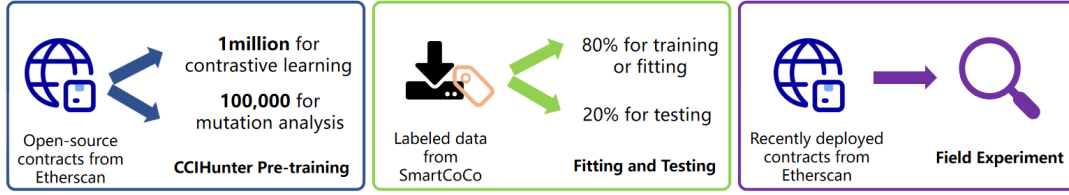


Fig. 7. The sources of data and their purposes used in the experiments

functions. From these instances, we randomly select 1 million function instances for contrastive learning and 100,000 function instances for mutation analysis. In the mutation analysis, 15 mutants are generated for each function.

The choice of data volume for pre_training is estimated according to the Scaling Laws proposed by Kaplan et al. [17]. According to their conclusions: during the pre-training of models, the performance penalty is predictably dependent on the ratio $\frac{N^{0.74}}{D}$. When the model size N is fixed and the data volume D is increased, or when the data volume D is fixed and the model size N is increased, the model enters a stage of diminishing returns, i.e., the performance penalty caused by overfitting. Therefore, we use $N^{0.74}$ to estimate the required data volume for pre-training. In the contrastive learning stage, we adjust the parameters of the graph embedding model (with parameter count $N_1 = 124,940,928$, approximately 120 million) and the text embedding model (with parameter count $N_2 = 2,904,224$, approximately 2.9 million). The total parameter count is about 128 million. Therefore, the estimated data volume is $(N_1 + N_2)^{0.74} = 997,576.83 \approx 1$ million. In the mutation analysis stage, we adjust the parameters of the graph embedding model, with a total parameter count of approximately 120 million. Therefore, the estimated data volume is $N_1^{0.74} = 980,757.18 \approx 980,000$. Considering that a large amount of training has already been conducted in the contrastive learning stage, we chose about 10% of the data volume (100,000) for the retraining of the graph embedding model in this stage, to avoid significantly altering the results obtained from the contrastive learning training.

For classifier fitting and testing, we adopt the dataset proposed by Hao et al. [15]. After removing contracts with low-version compilers, there are a total of 188 contracts, containing 9,857 commented function instances, among which the number of inconsistent function instances is 218 (hereinafter referred to as label_dataset). We have analyzed the transaction volumes of each contract and found that 52 out of 188 (27.66%) contracts have more than 50 transactions, 42 contracts (22.34%) have more than 100 transactions, and 19 contracts (10.17%) have more than 1,000 transactions. Therefore, it can be concluded that label_dataset includes contracts with high transaction volumes from the real world and thus is representative for testing. For the wild-detection experiment, on February 9th, 2025 we scraped 1000 recently deployed contracts provided by Etherscan [4]. We compile and model these contracts, and then conduct the wild-detection experiment. The sources of data and their purposes used in the experiments are shown in Fig. 7.

Metrics. The detection of CCIs is a binary classification task. To measure the detection performance of CCIHunter and other methods, we use the macro-averages of Precision, Recall, and F1 Score as evaluation metrics.

- **Precision:** Precision refers to the proportion of true positive samples among all samples predicted as positive by the model. It measures how effectively the model captures true positive instances.
- **Recall:** Recall is the proportion of true positive samples among all actual positive samples. It assesses how successfully the model captures true positive instances.
- **F1 Score:** The F1 Score is the harmonic mean of Precision and Recall. A high F1 Score indicates a well-balanced trade-off between precision and recall.

The formulas for these metrics are as follows:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where TP (True Positives) represents the number of samples that are actually positive and are correctly predicted as positive by the model, FP (False Positives) represents the number of samples that are actually negative but are incorrectly predicted as positive by the model, and FN (False Negatives) represents the number of samples that are actually positive but are incorrectly predicted as negative by the model. When assessing the method's performance on inconsistent data, we treat inconsistent code comment pairs as positive samples and consistent pairs as negative samples. The classification is reversed when evaluating the method on consistent data.

For the two classes of consistent data and inconsistent data, we calculate the above-mentioned metrics for each class and then take the average of the two classes (i.e., the macro-average) as the final evaluation metric. Such a metric can effectively evaluate the model's performance on imbalanced datasets.

Experimental settings. The prototype of CCIHunter is implemented in Python. Our experiments are conducted on a workstation with 512 GB of RAM, an Intel(R) Xeon(R) Gold 6148 CPU, and a GPU of NVIDIA GeForce RTX 3090 running on Ubuntu 20.04.1. Unless otherwise specified, we split the `label_dataset` at a ratio of 0.8:0.2 for the training and testing of CCIHunter and baselines other than the LLM respectively. For CCIHunter, the parameter settings are as follows: `epoch` = 100, `learning rate` = 0.001, `batch_size` = 64, `layers_num` = 6, and `attention_heads` = 6. Considering that the processes of the Random Forest, Logistic Regression, Decision Tree, and Gradient Boosting Classifier in the experiments involve random feature selection, we conduct the experiments 5 times and take the average value as the final experimental result.

5.1 RQ1: What is the better choice of the CCIHunter classifier and its parameters?

CCIHunter includes a classifier for classifying whether there is a CCI in a function. To answer RQ1, we explore which classifier is a better choice for CCIHunter and tune the hyperparameters of the classifier.

For the selection of the classifier, we compare eight common classifiers: Gaussian NB, KNeighbors Classifier, Support Vector Classifier, Logistic Regression, Decision Tree Classifier, Random Forest Classifier, Gradient Boosting Classifier, and Ada Boost Classifier. At this stage, all classifiers use default parameters. We present the performance of different classifiers in Table 1.

Table 1. Performance results of different classifiers

Classifier	Precision	Recall	F1
Gaussian NB	0.50	0.51	0.43
KNeighbors Classifier	0.49	0.50	0.49
Support Vector Classifier	0.49	0.50	0.49
Logistic Regression	0.49	0.50	0.49
Decision Tree Classifier	0.98	0.77	0.84
Random Forest Classifier	0.97	0.83	0.89
Gradient Boosting Classifier	0.91	0.67	0.75
Ada Boost Classifier	0.93	0.57	0.62

When using default parameters, the RandomForest Classifier achieves a precision of 0.97, a recall of 0.83, and an F1 score of 0.89, outperforming other classifiers. An interesting finding is that tree-based models (Decision Tree Classifier, Random Forest Classifier, Gradient Boosting Classifier, Ada Boost Classifier) perform better than other types of classification models (Gaussian NB, Logistic Regression, Support Vector Classifier, KNeighbors Classifier). This may be because the CCI detection task involves complex relationships among data features, and tree-based models can automatically learn the non-linear relationships and feature interaction information in the feature space formed by concatenating code vectors, comment vectors, and similarity vectors. Among the four tree-based classification models, the random forest model shows better performance when using default parameters. This might be because the random forest integrates the prediction results of multiple decision trees, enabling it to better handle noise and overfitting problems in the data. When dealing with imbalanced data, the random sampling and feature selection mechanisms of the random forest can, to some extent, alleviate the problem of minority-class samples being ignored. Based on the above analysis, we select the random forest classifier as the classifier for CCIHunter for subsequent experiments.

After selecting the random forest classifier as the classifier for CCIHunter, we further explore the optimal parameters for the classifier, that is, we perform hyperparameter tuning. Among the hyperparameters of the random forest classifier, 'n_estimators' represents the number of decision trees in the random forest; 'criterion' is the standard for measuring the quality of decision-tree node splitting, with three options: 'gini', 'entropy', and 'log_loss'. 'Gini' represents the Gini impurity, which measures the probability of a randomly chosen element being misclassified if it were randomly labeled according to the distribution of labels in the node. 'Entropy' is a measure of the impurity of a set of samples, based on the concept of information entropy from information theory. Lower entropy means the set is more homogeneous in terms of class labels. 'Log_loss' is also known as logarithmic loss, which is used to evaluate the performance of a classifier where the predicted output is a probability value, and it penalizes confident and wrong predictions more heavily. 'Max_features' refers to the maximum number of features considered when splitting each decision-tree node. It can be set to 'sqrt', which indicates that the maximum number of features to consider when looking for the best split is the square root of the total number of features; 'log2', which means the maximum number of features is the base-2 logarithm of the total number of features; or 'None', which implies that all features will be considered when splitting nodes.

We aim to find the optimal parameter combination of these three hyperparameters. Since the default value of 'n_estimators' is 100, we set the candidate parameter set for 'n_estimators' as [10, 50, 100, 150, 200, 250]. The three choices of 'criterion' and the three choices of 'max_features' mentioned above form 9 parameter combinations. We evaluate the performance of the random forest classifier under different parameter settings on the test set, and the results are shown in Fig. 8.

From the experimental results, setting 'max_features' to 'None' is the optimal choice compared to the other two options. This means allowing every decision-tree node to take all features into account when splitting. This outcome is as expected because the CCI detection task involves a complex feature space created by concatenating code vectors, comment vectors, and similarity vectors. There might be extensive interaction information and intricate non-linear relationships among different features. Considering all features enables the decision tree to more comprehensively explore the feature space, thus better capturing these complex relationships, discovering the potential patterns and rules in the data, and enhancing CCIHunter's capability to identify CCIs. Using the F1 score as the evaluation metric, on this dataset, the combination of n_estimators = 200, criterion = log_loss, and max_features = None is the best choice, achieving an F1 score of 0.93, which is superior to other parameter settings. Therefore, in subsequent experiments, we set the parameters of CCIHunter's classifier to n_estimators = 200, criterion = log_loss, and max_features = None.

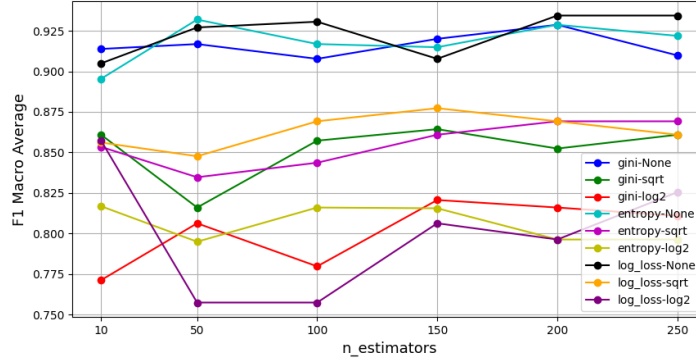


Fig. 8. The performance of random forest classifiers with different hyperparameters

Table 2. Comparative experimental results

Method	Inconsistent		Consistent		Macro-Average		
	Precision	Recall	Precision	Recall	Precision	Recall	F1 Score
Siamese	0.018	0.727	0.992	0.460	0.51	0.59	0.33
Topic	0.000	0.000	0.962	0.990	0.48	0.49	0.49
CodeBert	0.000	0.000	0.988	1.000	0.49	0.50	0.50
SmartCoCo	0.855	1.000	1.000	0.996	0.93	0.99	0.96
CCIHunter	0.905	0.806	0.995	0.998	0.95	0.90	0.93

5.2 RQ2: How does CCIHunter perform compared to existing methods?

To answer RQ2 and explore the effectiveness of CCIHunter, we conducted comparative experiments. As CCIHunter is learning-based, we select three machine learning-based method [13, 21, 22] as baselines. Specifically, Rabbi et al. proposed a method that uses a Siamese neural network [22], which is referred to as Siamese in this paper. Siamese embeds comment and code ASTs separately using RNN - LSTM, calculates the distance between them, and establishes consistency. Rabbi, F. et al. also proposed a method based on topic modeling [21], which is referred to as Topic in this paper. Topic extracts features from code and comments using topic modeling, merges consistency scores from random forests, and analyzes their consistency. Feng et al. [13] introduced CodeBERT, a dual-modal pre-training model for programming languages and natural languages, supporting various downstream tasks. Therefore, we leverage CodeBERT to generate embeddings for both of comments and code, facilitating the detection of inconsistencies. In addition to the aforementioned learning-based approaches, we also compare with the rule-based method SmartCoCo [15], which is the only existing CCI detection method in the smart contract domain. For Siamese, we use the parameter settings from its paper, with learning rate = 0.05 and epoch = 600K. For Topic, since the paper does not provide the corresponding parameter settings, we set the parameters of the SVM it uses to the default values. For Codebert, we directly use the pre-trained Codebert. The results of the comparative experiments are presented in Table 2. CCIHunter achieves a precision of 0.95, a recall of 0.90, and an F1 score of 0.93.

First, compared with previous learning-based methods (Siamese, Topic), the recall of CCIHunter is relatively increased by 52.54%, and the F1 score is relatively improved by 86%. The potential reasons for the poor performance

of existing learning-based methods may include: (1) In terms of method design, existing learning-based methods are not tailored for smart contracts, whereas CCIHunter analyzes the characteristics of smart contract CCIs and undergoes a two-stage pre-training process; (2) In terms of data, the performance of existing learning-based methods is reliant on labeled data. However, there is a limited amount of labeled data for Solidity CCIs. Moreover, in the experimental dataset, the ratio of non-CCI to CCI instances is significantly skewed, approximately 44:1. In contrast, previous works by Siamese and Topic were based on datasets with a ratio of 1.4:1 for training and testing, which is considerably more balanced. The markedly imbalanced data in the current dataset is likely to diminish the effectiveness of their methods. In real-world scenarios, since deploying contracts requires paying a certain gas fee, projects for real-world businesses usually undergo reviews before deploying. As a result, compared to functions without CCI, those containing CCI are in the minority, and the ratio between non-CCI and CCI is significantly skewed. Based on this understanding, it is unreasonable to design and conduct tests based on a balanced dataset.

Second, replacing the code embedding model with CodeBert does not yield good results (Precision: 0.48, Recall: 0.49, F1: 0.49). This may be because: (1) CodeBert has an input limitation, requiring the length of input tokens to be no more than 512. This limitation prevents some codes from being fully learned, resulting in incomplete code data; (2) For the input of CodeBert, we did not further process the code. That is, CodeBert only learns the content of the function itself without including the content of its calling functions, leading to a lack of context information. In contrast, the design of CCIHunter based on the graph model can effectively solve the problems mentioned here. On the one hand, CCIHunter is based on a heterogeneous graph neural network model and performs graph embedding on the AST+ corresponding to the code data without any graph - scale limitation, so the model can learn complete code data. On the other hand, due to the absence of input - scale limitation, we can expand the AST according to the call relationships between functions during data modeling, enabling the code data to contain more context information and solving the problem caused by nested intentions.

Third, compared with the rule-based method SmartCoCo, CCIHunter can reduce false positives to a certain extent. It is worth noting that the dataset used for comparative experiments is the labeled dataset proposed by Hao et al [15]. They obtained this labeled dataset by manually annotating a subset of functions that SmartCoCo identified as inconsistent. Therefore, all the inconsistent functions labeled in this dataset are detectable by SmartCoCo. So in the experimental results of SmartCoCo, the recall for the inconsistent class is 1, and the precision for the consistent class is 1, which in turn improves the F1 score. So in the experimental results of SmartCoCo, the recall for the inconsistent class is 1, and the precision for the consistent class is 1, which in turn improves the F1 score. Therefore, in the comparison with SmartCoCo, we focus on whether CCIHunter can reduce the false positives reported by SmartCoCo. In the inconsistent category, our accuracy is improved by 5%, which means we effectively reduce false positives. Furthermore, we use 80% of the data correctly identified by SmartCoCo as the training set and the data misclassified by SmartCoCo as the test set for evaluation. In the test set, SmartCoCo falsely reports 43 functions as inconsistent, whereas CCIHunter identifies only one of them as a function with inconsistent code comments. This means that, compared to SmartCoCo, CCIHunter can significantly reduce the false positive rate, which is crucial for developers. Unnecessary false positives may impose additional auditing costs on developers and can also diminish their confidence in the tool.

Considering that the acquisition method of the test set to some extent affects the performance of the comparative method SmartCoCo, we conduct a wild detection experiment to further make a fair comparison between our method and SmartCoCo. We crawl the latest 1,000 deployed contract information provided by Etherscan on February 9th, 2025 and perform inconsistency detection using both CCIHunter and SmartCoCo. For these 1,000 contracts, CCIHunter reports a total of 79 inconsistent functions, while SmartCoCo reports 96 inconsistencies (note that, SmartCoCo may report multiple inconsistencies for each function). Given the widespread code reuse in smart contracts, there may be overlaps in the inconsistent functions reported by CCIHunter and SmartCoCo. Therefore, we first deduplicate the detection results from both methods. By manually examining the reported

inconsistent functions and removing those with identical function code, CCIHunter reports 9 unique functions with CCI, while SmartCoCo reports 22 functions with 28 inconsistencies. We further conduct manual labeling to determine whether these functions truly contain CCI. The results show that out of the 9 inconsistent functions reported by CCIHunter, 6 are accurate; whereas out of the 22 functions reported by SmartCoCo, 7 functions with 7 inconsistencies are correct. Among the CCI functions correctly detected by both CCIHunter and SmartCoCo, 4 functions are identical, indicating that both tools effectively identify these four inconsistent functions. CCIHunter and SmartCoCo have similar performance in detecting correct CCI functions. However, SmartCoCo generates more false positives due to the following reasons: inability to extract code facts from if-statements (61.90%), failure to extract code facts from called functions (14.29%), inability to extract code facts from assembly language (9.52%), forced fact matching (9.52%), and language understanding issues (4.76%). In contrast, CCIHunter significantly reduces the false positive rate, which is consistent with the results of the previous experiments. In the total of 9 distinct functions with CCI detected by CCIHunter and SmartCoCo, 5 are related to fund transfers or authorizations (e.g., `transferFrom`, `safeTransferFrom`, `setApprovalForAll`, etc.). None of these functions fully implement the parameter constraints mentioned in the comments, potentially posing risks. Moreover, 77.78% of the functions have CCI due to the lack of parameter constraints, while the remaining functions have CCI due to the absence of restrictions on the function callers.

In addition, large language models (LLMs) are also one of the baselines we consider for comparison. A large language model refers to a deep-learning model with a large number of parameters, which is typically used for natural language processing tasks. The LLM learns language representations through large-scale training. In recent years, it has become increasingly popular and is widely used in various downstream tasks. Among them, DeepSeek [6] attracts significant attention due to its powerful reasoning ability and excellent performance on multiple tasks. Therefore, we also attempt to use DeepSeek for CCI detection. Following the comparison experiment between SmartCoCo and LLMs, we investigate the capability of DeepSeek in identifying and analyzing CCIs. We conduct experiments based on the official API [7] provided by DeepSeek, with the parameter specified as `model='deepseek-chat'` (corresponding to the DeepSeek-V3 version). We input 30 contracts containing functions with CCIs to DeepSeek, which are the same data used by Hao et al. [15] to test GPT-3.5 and Bing AI. These 30 contracts cover the three types of CCI proposed by SmartCoCo, namely, 10 related to role permissions, 10 related to parameter requirements, and 10 related to events. We require the LLM to detect and explain CCI for all functions in the tested contracts. The prompt we give to DeepSeek is as follows: “Assuming you are currently an auditor of smart contracts, please analyze the following smart contract code to determine if there is any code comment inconsistency (CCI) in the code, which means that the intent of the code implementation does not match the intent described in the comments. If CCI exists, please indicate its function name and provide an explanation.”

Among the 30 CCI cases, DeepSeek correctly detects 4 CCI cases, but only provides correct explanations for 2 of them. Based on these results, we find that directly conversing with an LLM for CCI detection does not yield good results, which is similar to the conclusion of Hao et al. [15]. The possible reason is that we do not provide sufficient background knowledge and samples for DeepSeek to learn and understand what CCI in smart contracts is, so DeepSeek may lack the learning of domain-specific knowledge. We believe that large language models have certain potential in detecting CCI and providing reasonable explanations, but they may need to be specifically fine-tuned or designed.

5.3 RQ3: How dependent is CCIHunter on labeled data?

To answer RQ3 and investigate the dependency of CCIHunter’s effectiveness in detecting CCIs on labeled data, we conducted experiments by partitioning the training and testing sets at different ratios and calculated the metric results for each detection. It is important to note that the labeled data used by CCIHunter is exclusively for fitting the classifier in the training set; no labeled data is required during the pre-training phase.

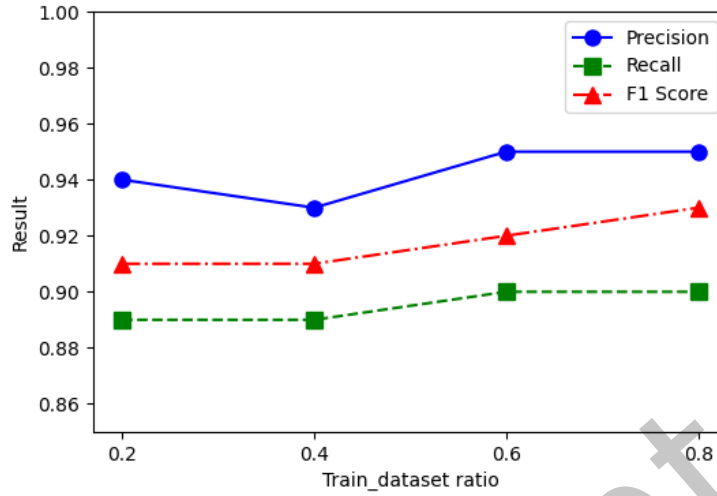


Fig. 9. The performance of CCIHunter under varying proportions of dataset

Table 3. Result of ablation experiments

data modeling	contrastive learning	mutation analysis	similarity feature	Precision	Recall	F1 Score
✓	×	×	✓	0.93	0.57	0.62
✓	✓	×	✓	0.95	0.62	0.68
✓	✓	✓	×	0.93	0.87	0.90
×	✓	✓	✓	0.83	0.74	0.79
✓	✓	✓	✓	0.95	0.90	0.93

Fig. 9 shows the performance of CCIHunter under varying proportions of the dataset. The x-axis represents the proportion of the training set in the total dataset (ranging from 20% to 80% in increments of 20%), and the y-axis represents the precision, recall and F1 score of CCIHunter. The results indicate that even with a training set comprising only 20% of the labeled data, CCIHunter still achieves a precision of 0.94 and a recall of 0.89. Moreover, the performance of CCIHunter varies minimally across different dataset ratios. The maximum difference in F1 scores under different dataset partitioning scenarios is approximately 0.2. This suggests that CCIHunter’s reliance on labeled data for detecting CCIs is comparatively modest, enabling it to address the challenge of limited labeled CCI dataset in smart contracts.

5.4 RQ4: Is the method design of CCIHunter effective?

To answer RQ4, we evaluate the specific impact of each component on the performance of CCIHunter through ablation experiments. In the design of CCIHunter, there are four key steps or components: data modeling, contrastive learning, mutation analysis, and similarity features. We design five ablation experiments to assess the effectiveness of each component. The component settings of the five experiments and the performance of CCIHunter are shown in Table 3, where “✓” indicates that the corresponding component is enabled, and “×” indicates that it is disabled.

(1) Effectiveness of data modeling: By comparing the results of the fourth and fifth experiments, it can be seen that the data modeling of CCIHunter can effectively improve its precision, recall, and F1 score. Specifically, the precision increases from 0.83 to 0.95, the recall increases from 0.74 to 0.90, and the F1 score increases from 0.79 to 0.93. This is because in the process of data modeling, we consider the possible problem of missing context information caused by comment references and intention nesting, and design corresponding data modeling methods (i.e., template-based comment modeling and function-call-based AST expansion) to supplement more context information. This context information can help CCIHunter learn more code facts and constraints of functions, thus making more accurate judgments.

(2) Effectiveness of two-stage pre-training: The introduction of contrastive learning and mutation analysis significantly improves the recall of the model. Compared with the results of the first experiment, in the second experiment, contrastive learning increases the recall from 0.57 to 0.62, and mutation analysis further boosts the recall to 0.87 in the third experiment. This indicates that these two components play an important role in enhancing the model's ability to recognize positive examples. This is because the two-stage pre-training process of CCIHunter can bring the distances of consistent code-comment pairs closer and separate the distances of inconsistent code-comment pairs. To further illustrate this point, we calculate the similarities (referred to as SimScore mentioned in Equ. 1) of consistent and inconsistent code-comment pairs with CCIHunter before and after training respectively. The results show that the average similarity between consistent code-comment pairs in the label_dataset before pre-training is 0.01334, which increases to 0.03074 after pre-training, an increase of 0.0174. While the average similarity between inconsistent code-comment pairs decreases from 0.01462 to -0.12653 after training, a decrease of 0.14115. It can be seen that before pre-training, the average similarities of consistent and inconsistent code-comment pairs are close. After pre-training, the average similarity of consistent code-comment pairs in the embedding space increases, and conversely, the average similarity of inconsistent code-comment pairs decreases, which meets the expectations of pre-training.

(3) Effectiveness of similarity features: The introduction of similarity features has a positive impact on both precision and F1 score. By comparing the results of the third and fifth experiments, it can be found that similarity features increase the F1 score from 0.90 to 0.93. This is because the similarity score reflects the consistency of intentions between code and comments to a certain extent, and thus can be an effective feature to help CCIHunter detect CCI.

In the fifth experiment, when all components are enabled, the F1 score reaches the highest value of 0.93, emphasizing the importance of the collaborative work of each component and their complementary roles in improving the model's precision and recall.

5.5 Case Study

We mention three motivating examples in the previous section to inspire the design of the CCIHunter method. Therefore, in the case study, we explore whether CCIHunter can effectively deal with these cases and further discuss the effectiveness of CCIHunter. Table 4 shows the information of these cases. We use CCIHunter to detect these three cases, and the results show that CCIHunter can accurately classify them. Specifically, it judges case2 as CCI and case1 and case3 as non - CCI. The excellent performance of CCIHunter on these three cases benefits from its method design. Specifically, it benefits from the two-stage pre-training and the data modeling that takes into account comment references and intention nesting. To verify this idea, we conduct further analysis.

First, we calculate the similarities between code and comment embeddings before and after pre-training for the three cases. The results are shown in Table 5. It can be observed that after pre-training, for case2 with code - comment inconsistency, the similarity between the code and comment embeddings generated by CCIHunter decreases from 0.0620 to -0.0588. Conversely, for case1 and case3 without CCI, the similarities between the code and comment embeddings generated by CCIHunter increase. Specifically, the similarity for case1 increases from

Table 4. Information of three cases

Case	Contract	Function	Label	Intent Nesting	Comment Reference
case1	PlayChip	transferFrom	non-CCI	✓	-
case2	GravisCollectible	transferFrom	CCI	✓	✓
case3	ContractRegistry	setContract	non-CCI	-	-

Table 5. SimScores between the code and comment using embedding models before and after pre-training

Case	label	SimScore before pre-training	SimScore after pre-training	Change
case1	non-CCI	0.0570	0.2679	↑ 0.2109 (+370%)
case2	CCI	0.0620	-0.0588	↓ 0.1208 (-194%)
case3	non-CCI	-0.0608	0.1144	↑ 0.1752 (+288%)

Table 6. Comparison of data modeling with and without considering comment references and nested intentions

Case	label	Modeling method 1		Modeling method 2		Similarity change
		Similarity	Determination	Similarity	Determination	
case1	non-CCI	0.0341	non-CCI	0.2679	non-CCI	↑ 0.2338
case2	CCI	0.0565	non-CCI	-0.0588	CCI	↓ 0.1153

0.0570 to 0.2679, with an increase of 0.2109, and the similarity for case3 increases from -0.0608 to 0.1144, with an increase of 0.1752. This is in line with our expectations and indicates that the pre-training achieves certain effects. Because the goal of pre-training is to generate higher-quality embeddings for code and comments, and to reduce the distance between consistent code - comment pairs in the embedding space while increasing the distance between inconsistent pairs.

Second, since there is intention nesting or comment referencing in case 1 and case 2, we employ two distinct methods for modeling them: one that disregards comment references and nested intentions, utilizing the original comments and function ASTs directly (Modeling method 1); and another that deals with comment references and nested intentions by template-based comment enhancement and function call-based AST expansion during data modeling (Modeling method 2). We then calculate the similarities of code comment embeddings under both modeling approaches, with the results presented in Table 6. It can be observed that our data modeling approach enables CCIHunter to correctly classify case1 and case2. Furthermore, the similarity between code and comment in case1, which does not contain CCI, is further enhanced from 0.0341 to 0.2679, while the similarity in case2, which contains CCI, decreases from 0.0565 to -0.0588. It is worth mentioning that for case2, which involves comment references and nested intentions, if data noise is not considered during data modeling, the model will fail to report the function with CCI. This aligns with our expectation that ignoring the data comment reference and nested intentions could lead to missed reports. In summary, the data modeling of CCIHunter is effective and can assist CCIHunter in correctly classifying case1 and case2.

Furthermore, it is worth noting that both case1 and case3 are instances where SmartCoCo has falsely identified CCIs. We have further analyzed the reasons for these false positives. In the instance of case1, SmartCoCo has extracted a code constraint suggesting that the ‘from’ value should be less than the ‘transfer quantity.’ This interpretation is actually incorrect. Firstly, the comparison should not be between ‘from’ and ‘transfer quantity,’ but rather between the ‘from’ account’s ‘balance’ and the ‘transfer quantity.’ Secondly, the comment indicates that line 10 in case1 is not a code constraint but an exceptional condition, implying that the actual code constraint

should be that the ‘from’ account’s balance should exceed the ‘transfer quantity.’ SmartCoCo’s extraction of constraints from comments did not take into account the contextual information of the comment. Furthermore, in the called function ‘safeSub,’ a check on the ‘balance’ and ‘transfer quantity’ has already been implemented, fulfilling the constraint mentioned in the comment, yet SmartCoCo failed to capture this code fact. Consequently, case1 has led to a misjudgment of the function. As for case3, SmartCoCo has extracted constraints and code facts related to emitting an event. Within the context of the comment for case3, the requirement is not for the function to emit an event named “corresponding.” However, SmartCoCo has performed a simplistic match on the event name, leading to a misjudgment. In summary, based on patterns, SmartCoCo has issues with not considering the contextual information and semantics of comments, and its rigid rules struggle to adapt flexibly to the varying comment styles of different developers. This situation highlights the advantages of CCIHunter: on one hand, CCIHunter treats the entire comment as input when embedding, and the language model it uses is capable of capturing the contextual information within the comment text, reducing the potential for misunderstanding the comments. On the other hand, CCIHunter does not prescribe specific rules; instead, it undergoes pre-training on a large number of smart contracts, which allows it to learn from various comments and code, and to comprehend the underlying business logic.

Overall, the analysis results in our case study are consistent with the findings in the motivating examples. First, intention nesting and comment references can affect the accurate judgment of CCI. Second, developers have a high degree of flexibility in expressing and implementing their intentions, and relatively rigid rules struggle to cope with this flexibility. As a result, rule-based methods may generate false alarms. In addition, based on these three cases, we have once again verified the effectiveness of CCIHunter.

5.6 Threats to Validity

5.6.1 Internal Validity. A potential threat to internal validity is the discrepancy arising from different embedding models. The embeddings of comments and codes are generated by a language model and a graph neural network model, respectively. Due to the differences in models, the generated embeddings are inconsistent in the distribution space. The similarity calculated directly using the embeddings of the two models may not accurately reflect the intention consistency between codes and comments. However, CCIHunter can address this internal threat to a certain extent. First, in the contrastive learning procedure, based on the loss function L_1 , we narrow the distance between the matching codes and comments. Essentially, through the optimization of the loss function, the two embedding models are trained to make the similarity between the embeddings generated for consistent code-comment pairs higher. Second, during the CCI detection process, we provide three features, namely, the embedding of codes, the embedding of comments, and the code-comment similarity, to the classifier instead of using the similarity score as the sole classification basis, which can also alleviate the threat brought by this problem.

Another potential threat to internal validity is the selection of parameters. In our work, the parameters related to model pre-training, such as learning rate and batch_size, are not experimentally compared. However, since we use the adaptive optimizer Adam, the learning rate can automatically adjust to an optimal level. In addition, based on CLIP experience [23], appropriately increasing batch_size can improve the effectiveness of model training. Consequently, we have set the batch size to 64, which is the maximum value that meets the storage limitations of our graphics cards. We cannot guarantee that these parameters are the most suitable values, but the cost of selecting the best values for these parameters through multiple pre-training comparisons is substantial, and it is not the focus of this paper, so we do not delve into the selection of these parameters in depth. Furthermore, from the experimental results, CCIHunter with the current parameters has achieved relatively ideal experimental outcomes.

5.6.2 External Validity. A threat to external validity may be the generalizability to other languages. In this paper, CCIHunter is only trained and tested on smart contracts written in Solidity. However, comment references and intention nesting also exist in other smart contract programming languages. The data modeling method and the two-stage pre-training idea of CCIHunter can be applied to the CCI detection of other smart contract programming languages.

Another potential threat to external validity stems from the cross-contract function calls. CCIHunter has not focused on and resolved the issues that may arise from cross-contract function calls during the detection of CCIs. The effectiveness of CCIHunter may be somewhat limited when dealing with CCIs related to cross-contract interactions. Considering cross-contract function calls may require more advanced program analysis techniques. However, based on the technical approach we outlined, the data modeling methods for both single-contract and cross-contract function calls are similar, hence incorporating the capability for cross-contract analysis will not significantly alter the technical stack of CCIHunter. We will consider incorporating cross-contract analysis techniques in future work to further expand the scope of inconsistency detection.

5.6.3 Construct Validity. The construct validity threat of CCIHunter may lie in the rationality of the model assumptions. The design of the CCIHunter method is based on a fundamental assumption that the similarity between embeddings of consistent code and comments is high. This assumption may not hold true in all cases, thus posing a potential threat to construct validity. However, on the one hand, Corazza et al. [11] have demonstrated that consistent code-comment pairs exhibit higher lexical similarity. On the other hand, our pretraining method, through the optimization of the loss function, enables consistent code-comment pairs to achieve higher similarity in the embedding space, which has also been proven in the ablation experiments.

5.6.4 Conclusion Validity. The class distribution of the dataset may pose a threat to the validity of our conclusions. In the comparative experiments conducted in this study, we used the labeled dataset provided by Hao et al. [15], where the ratio of non-CCI to CCI instances is approximately 44:1. An imbalanced dataset may pose a certain threat to the validity of the experimental conclusions. However, on the one hand, in real-world scenarios, deploying contracts requires paying gas fees, and projects targeting real business applications are usually reviewed. Therefore, functions containing CCIs are relatively rare, leading to a large proportion of non-CCI to CCI instances, making the data class imbalance reasonable. On the other hand, we employed macro-average as the evaluation metric for the experiments. Macro-average is commonly used for evaluating imbalanced datasets, as it can ignore the sample proportions of different classes and comprehensively assess the performance of various methods in detecting consistent and inconsistent data.

Another potential threat to the validity of our conclusions may stem from data contamination. Some of the data used for fitting and testing might overlap with the 3.57 million contracts in our pre-training dataset. There is a theoretical intersection between the SmartCoCo dataset and our dataset. However, this data contamination problem poses limited harm to the model. First, during the pre-training process, we do not provide any supervised signals, i.e., we do not use any labeled data. Therefore, there is no risk of label leakage. Second, in our two-stage pre-training process, we assume that a function's code and its comments are consistent. This assumption is not always true. However, according to the data from Hao et al. [15], only 1.71% (1,745/101,780) of the contracts contain CCIs. Therefore, even if we mistakenly treat some inconsistent code-comment pairs as consistent during pre-training, the frequency of such occurrences should be relatively low. Compared to the overall volume of data used in pre-training, these cases are negligible.

6 DISCUSSION

In this section, we aim to conduct some discussions related to CCI, provide our insights for future research work, and also offer some suggestions to smart contract developers.

(1) LLM represents a promising direction. LLMs have demonstrated outstanding performance in natural language understanding and analysis and have been widely applied in the field of software engineering. Although our experiments show that directly using conversations with LLMs for CCI detection does not yield satisfactory results, we still firmly believe that LLMs have certain potential. Through means such as prompt engineering, model fine-tuning, and few-shot learning, the performance of LLMs in the CCI detection task is expected to be significantly improved.

(2) Smart contract developers should maintain good commenting practices. In the wild detection experiments, all detected inconsistent functions result from the lack of implementation of some constraints mentioned in the comments. Among these, 55.56% of the functions are related to transfer. The presence of CCI may indicate potential security risks in the functions. Therefore, developers should maintain good commenting practices, clearly define function constraints, and carefully verify whether all constraints are implemented after completing the code. Additionally, developers should be mindful of outdated comments and check whether comments need to be updated after modifying the code.

7 CONCLUSION

In this paper, we investigated the issue of code-comment consistency in smart contracts and proposed a pre-training-based method, CCIHunter. To the best of our knowledge, this paper presented the first learning-based method tailored for smart contract CCIs. Our approach began by extracting code and comments, encapsulating comments within a prompt, and modeling the code as an AST-based heterogeneous graph. We then utilized the language model CodeBERT and the heterogeneous graph neural network model UniMP to obtain comment embeddings and code embeddings, followed by calculating the similarity between the obtained embeddings. Subsequently, the code embeddings, comment embeddings, and similarity scores were concatenated as a feature and fed into a classifier to determine the presence of CCIs. It was notable that CCIHunter accounted for the issue arising from comment references and nested intentions and implemented template-based comment enhancement and function call-based AST expansion during data modeling, injecting richer contextual information into code and comment data. Moreover, we proposed a two-stage pre-training method that enhanced CCIHunter's ability to detect CCIs through contrastive learning and mutation analysis, with the entire pre-training process being label-free. We demonstrated through experiments that CCIHunter outperformed existing tools and performed well even with a limited amount of labeled data. We also conducted ablation studies to validate the effectiveness and necessity of the two-stage pre-training and further explored the model's efficacy in our case study.

REFERENCES

- [1] 2022. *CVE-2022-25335 Detail*. <https://nvd.nist.gov/vuln/detail/CVE-2022-25335>
- [2] 2022. *DAppRadAr leaderboard*. <https://dappradar.com/rankings>
- [3] 2022. *Web3 developer report*. <https://www.alchemy.com/blog/web3-developer-report-q4-2022>
- [4] 2024. *Etherscan*. <https://etherscan.io/>
- [5] 2024. *Solc compiler*. <https://docs.soliditylang.org/en/v0.8.23/using-the-compiler.html>
- [6] 2025. *DeepSeek*. <https://chat.deepseek.com/>
- [7] 2025. *DeepSeek API*. <https://api-docs.deepseek.com/api/deepseek-api>
- [8] Oliver Arafat and Dirk Riehle. 2009. The commenting practice of open source. In *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications*. Association for Computing Machinery, New York, NY, USA, 857–864.
- [9] Morena Barboni, Andrea Morichetta, and Andrea Polini. 2021. Sumo: A mutation testing strategy for solidity smart contracts. In *2021 IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, Madrid, Spain, 50–59.
- [10] Xiangping Chen, Peiyong Liao, Yixin Zhang, Yuan Huang, and Zibin Zheng. 2021. Understanding Code Reuse in Smart Contracts. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, Honolulu, HI, USA, 470–479.
- [11] Anna Corazza, Valerio Maggio, and Giuseppe Scanniello. 2015. On the Coherence between Comments and Implementations in Source Code. In *2015 41st Euromicro Conference on Software Engineering and Advanced Applications*. 76–83. <https://doi.org/10.1109/SEAA.2015.20>

- [12] Sergio Cozzetti B. de Souza, Nicolas Anquetil, and Káthia M. de Oliveira. 2005. A Study of the Documentation Essential to Software Maintenance. In *Proceedings of the 23rd Annual International Conference on Design of Communication: Documenting & Designing for Pervasive Information*. Association for Computing Machinery, New York, NY, USA, 68–75.
- [13] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. Codebert: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, Virtual Event, 1536–1547.
- [14] Beat Fluri, Michael Würsch, Emanuel Giger, and Harald C. Gall. 2009. Analyzing the Co-Evolution of Comments and Source Code. *Software Quality Journal* 17 (2009), 367–394.
- [15] Sicheng Hao, Yuhong Nan, Zibin Zheng, and Xiaohui Liu. 2023. SmartCoCo: Checking Comment-Code Inconsistency in Smart Contracts via Constraint Propagation and Binding. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Luxembourg, 294–306.
- [16] Walid M. Ibrahim, Nicolas Bettenburg, Bram Adams, and Ahmed E. Hassan. 2012. On the relationship between comment update practices and Software Bugs. *Journal of Systems and Software* 85 (2012), 2293–2304.
- [17] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeff Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *ArXiv abs/2001.08361* (2020). <https://api.semanticscholar.org/CorpusID:210861095>
- [18] Zhiyong Liu, Huanchao Chen, Xiangping Chen, Xiaonan Luo, and Fan Zhou. 2018. Automatic Detection of Outdated Comments During Code Changes. In *2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC)*, Vol. 01. 154–163. <https://doi.org/10.1109/COMPSAC.2018.00028>
- [19] Wanrong Ouyang and Baojian Hua. 2021. 'R: Towards Detecting and Understanding Code-Document Violations in Rust. In *2021 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, Wuhan, China, 189–197.
- [20] Sheena Panthaplackel, Junyi Li, Milos Gligoric, and Raymond J. Mooney. 2021. Deep Just-In-Time Inconsistency Detection Between Comments and Source Code. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021*. AAAI Press, Virtual Event, 427–435.
- [21] Fazle Rabbi, Md Nazmul Haque, Md Eusha Kadir, Md Saeed Siddik, and Ahmedul Kabir. 2020. An Ensemble Approach to Detect Code Comment Inconsistencies using Topic Modeling.. In *The 32nd International Conference on Software Engineering and Knowledge Engineering, SEKE 2020*. KSI Research Inc., Virtual Event, 392–395.
- [22] Fazle Rabbi and Md Saeed Siddik. 2020. Detecting Code Comment Inconsistency using Siamese Recurrent Network. In *2020 IEEE/ACM 28th International Conference on Program Comprehension (ICPC)*. ACM, Seoul, Republic of Korea, 371–375.
- [23] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning*. PMLR, Virtual Event, 8748–8763.
- [24] Inderjot Kaur Ratol and Martin P. Robillard. 2017. Detecting fragile comments. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Computer Society, Urbana, IL, USA, 112–122.
- [25] Felice Salviulo and Giuseppe Scanniello. 2014. Dealing with Identifiers and Comments in Source Code Comprehension and Maintenance: Results from an Ethnographically-Informed Study with Students and Professionals. In *18th International Conference on Evaluation and Assessment in Software Engineering, EASE '14*. Association for Computing Machinery, New York, NY, USA, 48:1–48:10.
- [26] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjing Wang, and Yu Sun. 2021. Masked Label Prediction: Unified Message Passing Model for Semi-Supervised Classification. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021*. ijcai.org, Virtual Event, 1548–1554.
- [27] Daniela Steidl, Benjamin Hummel, and Elmar Juergens. 2013. Quality analysis of source code comments. In *2013 21st International Conference on Program Comprehension (ICPC)*. IEEE Computer Society, San Francisco, CA, USA, 83–92.
- [28] Lin Tan, Ding Yuan, Gopal Krishna, and Yuanyuan Zhou. 2007. /*comment: Bugs or Bad Comments?*/. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles 2007, SOSP 2007*. ACM, New York, NY, USA, 145–158.
- [29] Lin Tan, Ding Yuan, and Yuanyuan Zhou. 2007. Hotcomments: How to Make Program Comments More Useful?. In *Proceedings of the 11th USENIX Workshop on Hot Topics in Operating Systems*. USENIX Association, San Diego, California, USA.
- [30] Lin Tan, Yuanyuan Zhou, and Yoann Padioleau. 2011. aComment: mining annotations from comments and code to detect interrupt related concurrency bugs. In *2011 33rd International Conference on Software Engineering (ICSE)*. ACM, Honolulu, HI, USA, 11–20.
- [31] Shin Hwei Tan, Darko Marinov, Lin Tan, and Gary T. Leavens. 2012. @tComment: Testing Javadoc Comments to Detect Comment-Code Inconsistencies. In *2012 IEEE Fifth International Conference on Software Testing, Verification and Validation*. IEEE Computer Society, Montreal, QC, Canada, 260–269.
- [32] Fengcai Wen, Csaba Nagy, Gabriele Bavota, and Michele Lanza. 2019. A Large-Scale Empirical Study on Code-Comment Inconsistencies. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. IEEE / ACM, Montreal, QC, Canada, 53–64.

- [33] Zhiying Wu, Jieli Liu, Jiajing Wu, Zibin Zheng, Xiapu Luo, and Ting Chen. 2023. Know Your Transactions: Real-time and Generic Transaction Semantic Representation on Blockchain & Web3 Ecosystem. In *Proceedings of the ACM Web Conference 2023*. Association for Computing Machinery, New York, NY, USA, 1918–1927.
- [34] Zhengkang Xu, Shikai Guo, Yumiao Wang, Rong Chen, Hui Li, Xiaochen Li, and He Jiang. 2024. Code Comment Inconsistency Detection Based on Confidence Learning. *IEEE Transactions on Software Engineering* 50, 3 (2024), 598–617. <https://doi.org/10.1109/TSE.2024.3358489>
- [35] Jianfei Zhou, Tianxing Jiang, Haijun Wang, Meng Wu, and Ting Chen. 2023. DAppHunter: Identifying Inconsistent Behaviors of Blockchain-based Decentralized Applications. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, Melbourne, Australia, 24–35.
- [36] Chenguang Zhu, Ye Liu, Xiuheng Wu, and Yi Li. 2023. Identifying Solidity Smart Contract API Documentation Errors. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, New York, NY, USA, 56:1–56:13.